Registration Failure Investigation

Result

The exact message, `We could not complete your registration. Please try again.`, is added in two places in `Controllers/AccountController.cs`:

- After `UserManager.AddToRoleAsync(student, "Student")` returns an unsuccessful `IdentityResult` (around lines 59–65). That branch rolls back the transaction and logs only the returned Identity error codes.
- In the `catch (Exception exception)` around the registration transaction (around lines 70–77). That branch logs the exception, then returns the generic message.

The repository contains no runtime log entry, exception details, or database state from the reported attempt. Therefore it is not possible to determine from available evidence whether the observed message came from the role-assignment failure or an exception, nor to confirm a specific root cause. No code fix was made because selecting one would be speculative.

Traced registration path

1. `RegisterViewModel` and MVC model validation run first. Required name/email/password fields, email format and length, password length (6–100), matching confirmation, phone format, and accepted terms are checked. Invalid model state returns the form without entering the `try` block or displaying the generic message.
2. The action checks for an existing email. A match returns the specific message `An account with this email already exists.`
3. Inside a database transaction, `GetNextStudentUserNameAsync` executes `SELECT NEXT VALUE FOR dbo.StudentUsernameSequence` and checks for a username collision.
4. `UserManager.CreateAsync(student, model.Password)` validates Identity password and user requirements and persists the user. A failed `IdentityResult` rolls back and maps returned error codes into ModelState; this path does not display the generic message.
5. `UserManager.AddToRoleAsync(student, "Student")` persists the role link. A failed result rolls back, logs the role assignment error codes, and displays the generic message.
6. If role assignment succeeds, the transaction commits, the username is put in TempData, and the action redirects to Login. Registration does not generate or validate account setup tokens.

`Program.cs` configures Identity with the EF Core store and default token providers plus the separate `AccountSetupTokenProvider`. The separate setup provider is used by administrator-created accounts only; it is not called by registration and does not alter this registration path.

Database and migration evidence

`ApplicationDbContext` defines the `StudentUsernameSequence`; migration `20261021134452_StudentUsernameSequence` creates it. The `AdminRoleSeed` migration inserts the `Student` role. This establishes what the checked-in migrations intend to create, but does not establish which migrations have been applied to the database used by the failed attempt. Startup automatically migrates only when `app.Environment.IsDevelopment()`; non-development deployments must have schema/roles provisioned separately.

User creation and role assignment occur inside the same explicit EF database transaction. The Identity result-failure branches explicitly roll back; exceptions unwind through async transaction disposal. No evidence is available to determine the persistence outcome if the failure happened during transaction commit or whether any earlier account from the reported attempt exists.

Actual failure and missing evidence

The source establishes the two possible generic-message branches but not which one ran. The exception handler already logs the exception object and stack trace with `logger.LogError(exception, "Student registration failed.")`. The role failure branch logs its error codes. Neither log entry is present in this workspace.

The safest next diagnostic is to retrieve the server/application log entry for the failed attempt, redacting user identifiers and all secrets, and determine whether it is the registration exception log or the role-assignment error-code log. If it is an exception, its exception type and sanitized message/stack are needed. Also verify the applied migration level and presence of the Student role/username sequence on the configured database using read-only inspection. Do not run migrations as part of that diagnostic without approval.

Changes and verification

- No application code was changed because the root cause is not confirmed and changing the catch behavior or schema assumptions would conceal or guess at the failure.
- `REGISTRATION_FAILURE_FIX.md` documents this investigation and the exact evidence needed to select a fix.
- `dotnet build Manasa.csproj --no-restore` initially could not finish because the running application held `bin/Debug/net9.0/Manasa.exe` open (MSB3021/MSB3027).
- `dotnet build Manasa.csproj --no-restore -p:UseAppHost=false -p:OutputPath=.\obj\RegistrationVerify\` succeeded with 0 warnings and 0 errors. No test project was identified in the checkout.
- Runtime verification of successful registration, invalid form handling, and partial-account behavior could not be performed: no safe database/test fixture or logs from the failing instance are available. No migrations or destructive database operations were run.